

SUPPLEMENTARY PROBLEM: VALID PALINDROME

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

NOTE:

- **YOU DO NOT NEED TO PRESENT THE INTUITION SECTION. IT IS PURELY THERE FOR US TO PROVIDE QUICK NOTES REGARDING HOW WE SOLVED THESE PROBLEMS**
- We are using 1-based array indices

1 Problem 1: Valid Palindrome

1.1 Problem Statement

Given a sequence of lowercase characters, return **TRUE** if the sequence is a palindrome; otherwise, return **FALSE**. A palindrome is defined as a sequence, $S = \langle s_1, \dots, s_n \rangle$, that reads the same forward and backward, i.e. $s_i = s_{n-i+1}$ for $1 \leq i \leq \lfloor n/2 \rfloor$ where n is the length of the sequence. Assume the strings do not contain any white spaces. Here are some examples

- tacacat
- wasitacaroracatisaw

1.2 Intuition

The nature of a palindrome constrains the way this problem can be approached. The first solution that comes to mind is to use two indices: one starting from the front and one from the end. As we check the elements at these indices, one index moves forward and the other moves backward.

However, we do not necessarily need two variables to hold the indices of the elements being checked, because knowing the index of one element gives you the index of the other (i and $n - i$). This variable will start at the very beginning of the string, $i = 1$, and moves inward as long as there is still a possibility that the sequence is a palindrome.

At each value of i , the algorithm checks whether the elements at the two corresponding indices are equal and proceeds accordingly. If we reach a point where the two pointers are equal or have crossed, then we know the sequence is a palindrome.

1.3 Pseudocode

Algorithm 1 Valid Palindrome

Require: $S = \langle s_1, \dots, s_n \rangle$

Require: S contains only lowercase letters and no whitespace

```
1: procedure ISPALINDROME( $S$ )
2:    $i \leftarrow 1$ 
3:   while  $i \leq \lfloor \frac{\text{length}(S)}{2} \rfloor$  do
4:     if  $s_i \neq s_{n-i+1}$  then
5:       return False
6:     else
7:        $i \leftarrow i + 1$ 
8:     end if
9:   end while
10:  return True
11: end procedure
```

1.4 Proof of Correctness

Note: The use of “<, >” is to indicate concatenation.

1.4.1 Loop Invariant

At the start of each iteration, the substring $\langle S[1..i-1], S[n-i+2..n] \rangle$ is a palindrome, meaning $S[1..i-1] = S[n-i+2..n]$.

1.4.2 Initialization

Before the loop starts, i is initialized to 1 (first element in array), so the substring in question is composed of ranges which are invalid. Therefore, substring does not exist, so the L.I. is trivially true.

1.4.3 Maintenance

Assume the L.I. is true for some iteration i . We check if the element i can extend the bounds of what we consider the palindrome to be. There are only two cases to consider.

1. $S[i] \neq S[n-i+1]$
 - We have broken the rules set in place by the palindrome definition. Therefore, stop iterating and correctly return false.
2. $S[i] = S[n-i+1]$
 - The algorithm verifies that the palindrome definition is still being followed. Therefore, we move the bound inward by incrementing i . Because the characters are equal, at the next iteration $\langle S[1..(i+1)-1], S[n-(i+1)+1..n] \rangle$, so the L.I. will be true.

1.4.4 Termination

First, we need to prove that this loop always terminates. The index i is initialized to 1 and incremented by 1 in each iteration, while the loop condition is $i \leq \lfloor \frac{\text{length}(S)}{2} \rfloor$. Since i increases by 1 each time and the length of S is finite, the loop can only execute a finite number of steps before terminating. Therefore, the loop always terminates.

When the loop terminates (loop termination condition), $i = \lfloor \frac{\text{length}(S)}{2} \rfloor + 1$. We now have to consider two cases:

If the length of the sequence is odd, $i = n - i + 1$, then substituting this fact into the L.I. gives $\langle S[1..n - i], S[n - i + 2..n] \rangle$ is a palindrome. It might seem as if we didn't accomplish the goal of proving the entire sequence of being a palindrome because element $S[n - i + 1]$ isn't accounted for. However, given the statement above, inserting a single element in between the subsets won't cause the palindrome property to fail, since a sequence of one character is always a palindrome.

Otherwise if the length of the sequence is even, then $i > n - i + 1$ which means we already checked this pair of elements during the last iteration of the while loop. Therefore, the entire sequence has been verified as a palindrome.

1.5 Runtime Analysis

The worst case for the algorithm occurs when the subsequence is a palindrome, otherwise the algorithm terminates early when a match isn't detected. All the operations in the body of the while loop complete in constant time. Therefore, the runtime of the algorithm is dependent upon the number of iterations of the while loop.

We only need to move our pointer halfway through the palindrome in this problem, otherwise we would be double checking elements. Since there are $\lfloor \frac{\text{length}(S)}{2} \rfloor$ iterations of the loop, worst case is $O(n)$ where n is the length of the subsequence.

$$O(n)$$